

Exercise detection

Physical exercise detection on an embedded node device

Problem statement

The goal of this project is to implement a physical exercise detection AI model running on an embedded node device.

The model should be able to detect and classify different types of physical exercises, mostly revolving around the upper body, as the device will be worn on the wrist (like a smartwatch). These exercises include, but are not limited to:

- Bicep curls
- Tricep extensions
- Shoulder presses
- Lateral raises
- etc.

The model should be able to run in real-time and provide accurate results.

Data collection

Apart from already available datasets, such as:

- [PAMAP2 Dataset](#)
- [Exercise Recognition from Wearable Sensors](#)

Data will also be collected using the target device. The target device is a [Arduino Nano 33 BLE Sense Rev2](#), with a built-in 9-axis IMU.

Sensors used

The model will primarily use data from the built-in 9-axis IMU, which includes:

- Accelerometer: Measures linear acceleration in three axes (X, Y, Z)
- Gyroscope: Measures angular velocity in three axes (X, Y, Z)

- Magnetometer: Measures magnetic field strength in three axes (X, Y, Z)

While it is expected that the accelerometer alone will be sufficient for detecting and classifying the exercises, the gyroscope and magnetometer data will also be collected and can be used to improve the model's performance if necessary.

Model development

The model will be implemented using TensorFlow Lite for Microcontrollers, which is a lightweight version of TensorFlow designed for embedded devices.

The model will be trained on the collected data and optimized for real-time inference on the target device. The model type and training process will be chosen among 3 options, which will also be compared for performance, accuracy, but also size and inference time. The options are:

- A simple feedforward neural network
- A convolutional neural network (CNN)
- A recurrent neural network (RNN)

Expected limitations and challenges

- Detecting an exercise falsely: this is a common problem in exercise detection, as the model may misclassify a non-exercise movement as an exercise. An option where a start/stop button is used to indicate the start and end of an exercise series can be implemented to mitigate this issue.
- Limited computational resources: the target device has limited computational resources, which may limit the complexity of the model and the amount of data that can be processed in real-time. This will require careful optimization of the model and the training process to ensure that it can run efficiently on the target device. Therefore it is not explicitly said how many exercises the model will be able to detect.
- Data collection: collecting a sufficient amount of data for training the model can be time-consuming and may require multiple participants to perform the exercises in order to capture a wide range of movements and variations (in case where the available datasets are not sufficient)

Goals and deliverables

- Data collection and preprocessing scripts
- Trained AI model for physical exercise detection
- Code for running the model on the target device

- Documentation and report on the project, including model performance and comparison of different model types and training processes.